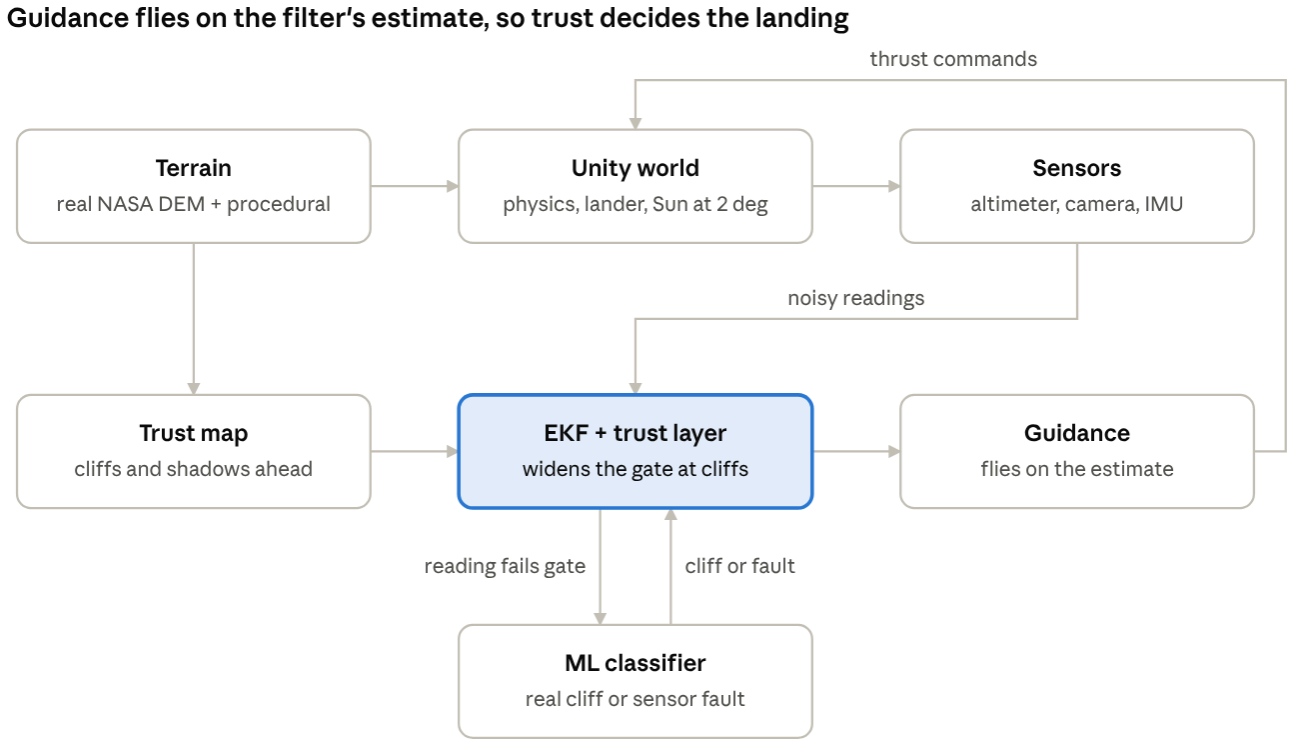
PRJ1 Proposed Methodology: Sensor Trust for Lunar South Pole Landing

We will build a physics-based lunar landing simulator in Unity on real NASA South Pole terrain. A lander there must navigate with simulated sensors, and our Context-Aware Sensor Trust Layer decides how much to believe each sensor. Success is whether the lander actually touches down upright, compared with a standard filter and a PITFAL-style filter (Adams et al., AIAA SciTech 2025).

At a glance



system architecture · 7 components, one control loop

The trust map is built from terrain and Sun position before descent. In flight, the EKF uses it to set its gate and camera weight, and asks the ML classifier only when a reading still looks wrong.

1. The trust layer: what changes in the filter

The main rule: four steps

For every altimeter reading, the trust layer asks four questions in order.

1. **Is it physically possible?** Find the lowest and highest ground on the map within the area where the lander might be. If the reading fits that range plus a small margin, go to step 2. If it fits nowhere on the map, it is a fault: reject it.
2. **Probation.** A surprising but possible reading is neither believed nor rejected at once. For 2–3 seconds the lander watches the next readings. If they agree with each other and follow the map's shape, it is real terrain: accept it and reset the height to match. If they jump around, it is a fault: reject it.
3. **Camera and shadow.** The map and the Sun's position say whether the ground below is lit. In darkness the camera is not used; in partial light it is used with less trust.
4. **No map coverage.** Outside the map, use the standard fixed rule plus probation.

Steps 1 and 2 address HAKUTO-R: its readings were possible given the crater and stayed consistent for minutes. Steps 2 and 3 address IM-2: noisy readings fail probation, and a camera in shadow is set aside.

The rest of this section explains the standard rule this replaces, the alternatives we compare against, and how site changes are handled.

A standard EKF accepts an altimeter reading only if its innovation is small compared with what the filter expects:

$$\nu = z - h(\hat{x}), \qquad S = HPH^T + R, \qquad \text{accept if } \nu^2/S < \chi^2_{\text{limit}}$$

The trust layer adds map-driven terms, computed along the planned corridor before descent:

$$S_{\text{alt}} = HPH^T + R_{\text{alt}} + \sigma^2_{\text{terrain}}(\hat{x}, P) + \sigma^2_{\text{map}}(\hat{x}), \qquad R_{\text{cam}} = \frac{R_{\text{cam},\,0}}{\max(\varepsilon,\ \text{lit fraction})}$$

- σ²_terrain is the variance of DEM heights inside the lander's horizontal uncertainty ellipse. It is near zero on flat ground, so the gate stays tight. At a crater rim with ±100 m uncertainty it reaches kilometres squared, so the gate opens where HAKUTO-R needed it to.
- σ²_map is the LOLA height error from the PGDA `_toterr` map.
- The lit fraction of the camera footprint is predicted from Sun geometry and the DEM, and measured from the rendered image in flight.
- **Fault-or-terrain check:** when a reading still fails the gate, an ML classifier decides whether nearby terrain explains it (accept and re-centre) or not (reject as a fault).

Ways of judging a surprising reading

The testbed accepts any "judge", so we compare several on the same descents:

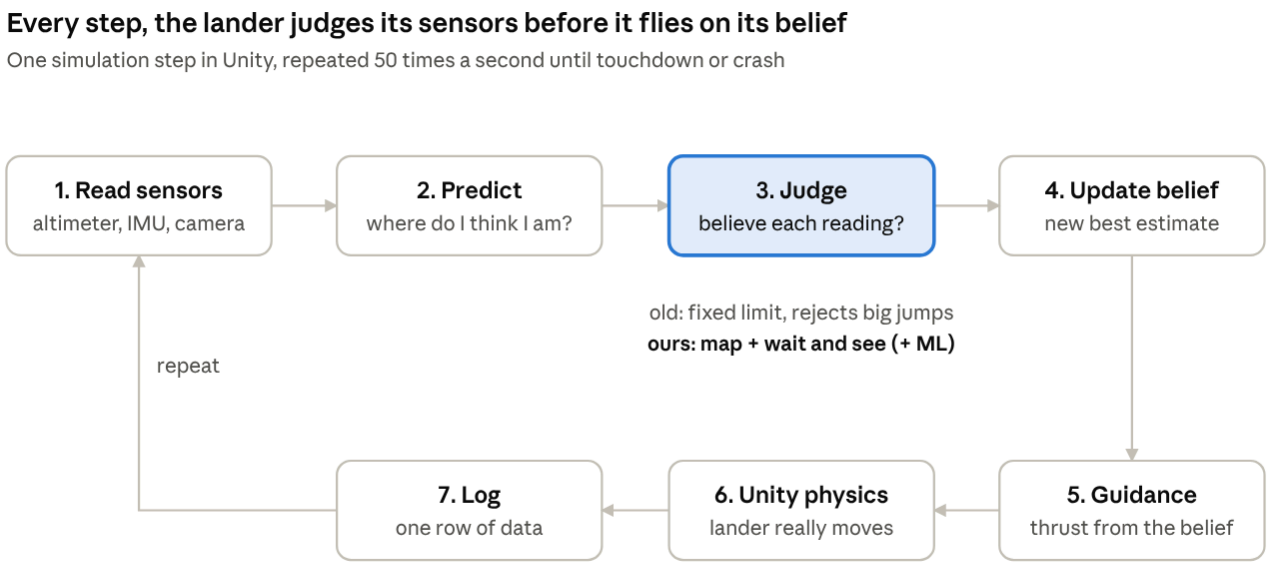
Judge	Idea	Example
Fixed rule (baseline)	Reject anything beyond a fixed chi-square limit	What HAKUTO-R did
PITFAL-style	Map in the prediction; fixed 99% limit; inflate noise after a flag	NASA/Draper, SciTech 2025
Map tolerance (ours)	Allow bigger surprises where the map shows height varying within the position uncertainty	"A 3 km drop is mapped here, so accept it"
Wait and see (ours)	A random glitch fades; a real cliff persists. Accept a surprise that stays consistent for a few seconds	HAKUTO-R's altimeter read ~5 km consistently for minutes
Second opinion (ours)	Check the surprise against the IMU and camera	Altimeter says "dropping" but the IMU feels no fall: a cliff, not a fall
ML classifier (ours)	A random forest trained on simulated descents labels each surprise as terrain or fault	Learns patterns the hand-written rules miss

The main rule above combines a physical-possibility check with probation (wait and see); the other judges are compared against it on the same landings.

Site changes

Trust is computed onboard from the map around the current estimated position, not only along the planned route, so a divert or a late site change needs no re-planning. Where no map exists, the filter switches to a cautious mode: moderately wider gates and more weight on cross-checks between sensors.

One simulation step



one simulation step · 7 stages

Guidance acts on the belief from step 4, so a wrong judgement in step 3 becomes a real crash in step 6. Step 7 records truth and belief side by side for the evaluation.

2. System architecture and tools

Layer	What it does	Tools
Terrain	Real PGDA DEMs converted to Unity heightmaps; procedural craters, boulders and roughness below 5 m; procedural crater worlds for training	Python, `rasterio`, `numpy`; Unity Terrain and mesh generation
Lighting	Directional Sun at 1.5° to 3° with real-time cast shadows; a wide horizon map so distant ridges cast shadows correctly	Unity HDRP; PGDA South Pole mosaic for the horizon
Physics	Rigid-body lander under lunar gravity (1.62 m/s²); throttleable engine, attitude thrusters, fuel budget; legs with joints so the lander can tip over	Unity PhysX; floating origin for 15+ km scales

Layer	What it does	Tools
Sensors	Laser altimeter as a raycast; IMU from rigid-body motion plus bias and noise; camera as a rendered image; fault injection (bursts, dropouts, delays)	C# scripts
Navigation	EKF with the trust layer; guidance flies on the estimated state, not the truth	C# at runtime; Python prototypes
ML	Fault-or-terrain classifier; crater detector on rendered images; camera reliability model; RL landing controller as an experiment	scikit-learn, PyTorch, ONNX, Unity Sentis, Unity ML-Agents (PPO)
Visualisation	HUD with true vs estimated altitude, live gate width, per-sensor trust bars; ghost lander at the estimated position; side-by-side replays	Unity UI

Guidance uses the filter's estimate, so a wrong altitude estimate produces a real crash in the physics engine, as on HAKUTO-R.

3. Dataset

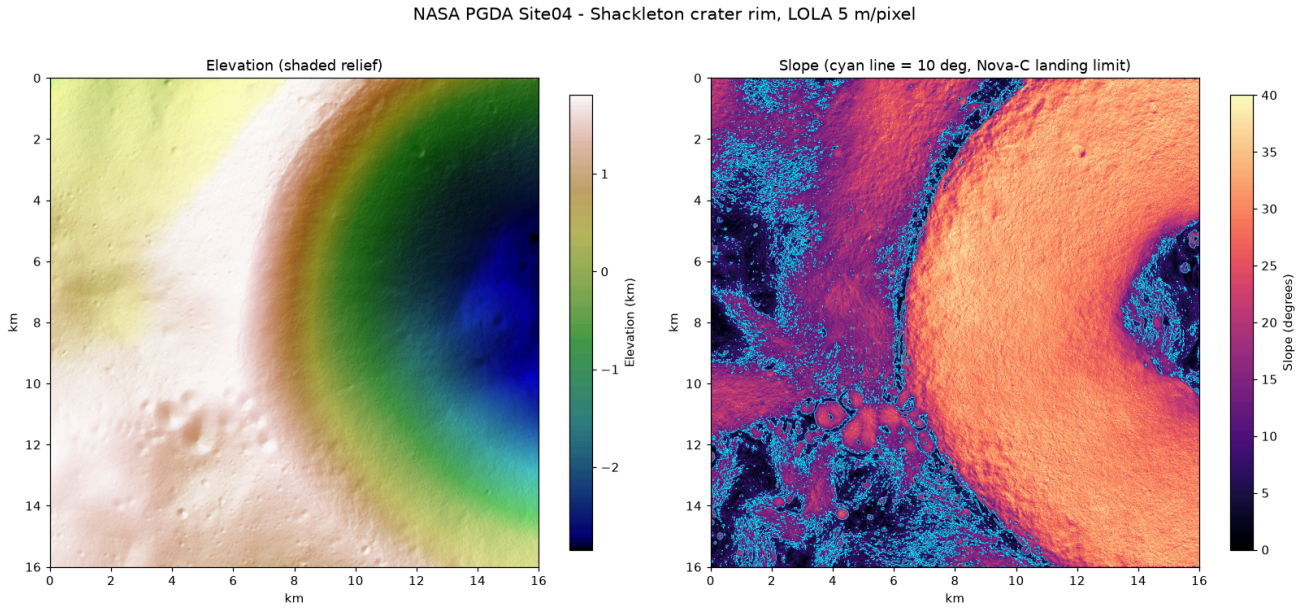
- **Real terrain:** NASA PGDA Product 78 (Barker et al.), Site04 Shackleton rim and Site01 connecting ridge: 5 m/pixel, 16 x 16 km tiles with elevation, slope and height-error maps.
- **Higher-detail terrain (optional):** PGDA Product 104 (Bertone et al.), Shape-from-Shading 5 m DEMs; Zenodo record 17954508, released 16 Dec 2025, about 2.2 GB per region.
- **Horizon map for shadows:** a wider, coarser South Pole LOLA mosaic around each tile.
- **Synthetic data:** every simulated run records sensor readings, true state, injected faults and rendered images. ML labels come free from the simulator's ground truth.
- **Mission reference profile:** the published HAKUTO-R M1 timeline (rangefinder updates from about 15 km; a gap of over 3 km rejected) scripts a reproduction scenario.

Dataset check (16 Oct 2026)

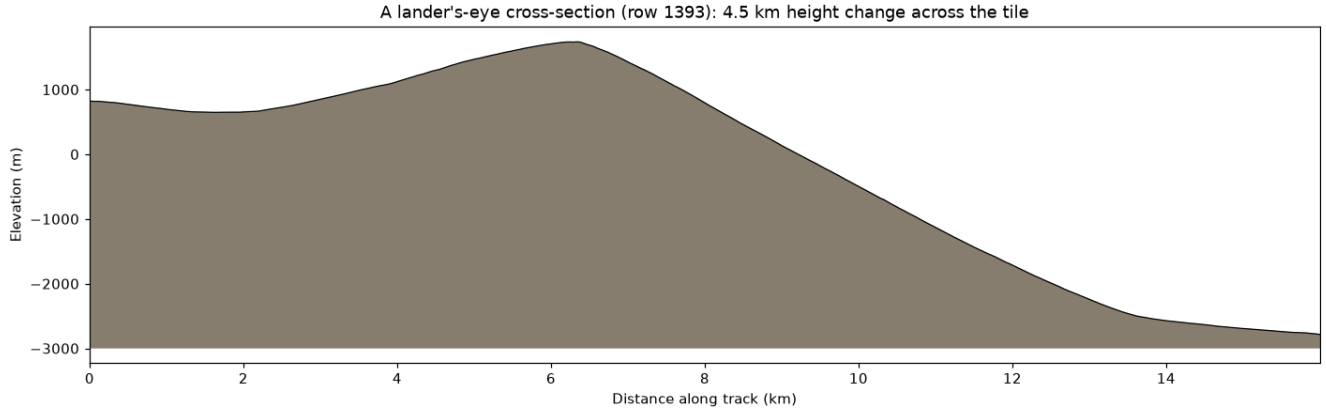
We downloaded and rendered Site04 to confirm it supports the plan.

- The tile is 3,200 x 3,200 pixels at 5 m and spans −2,848 m to +1,805 m (4.65 km of relief), with slopes up to about 70°.

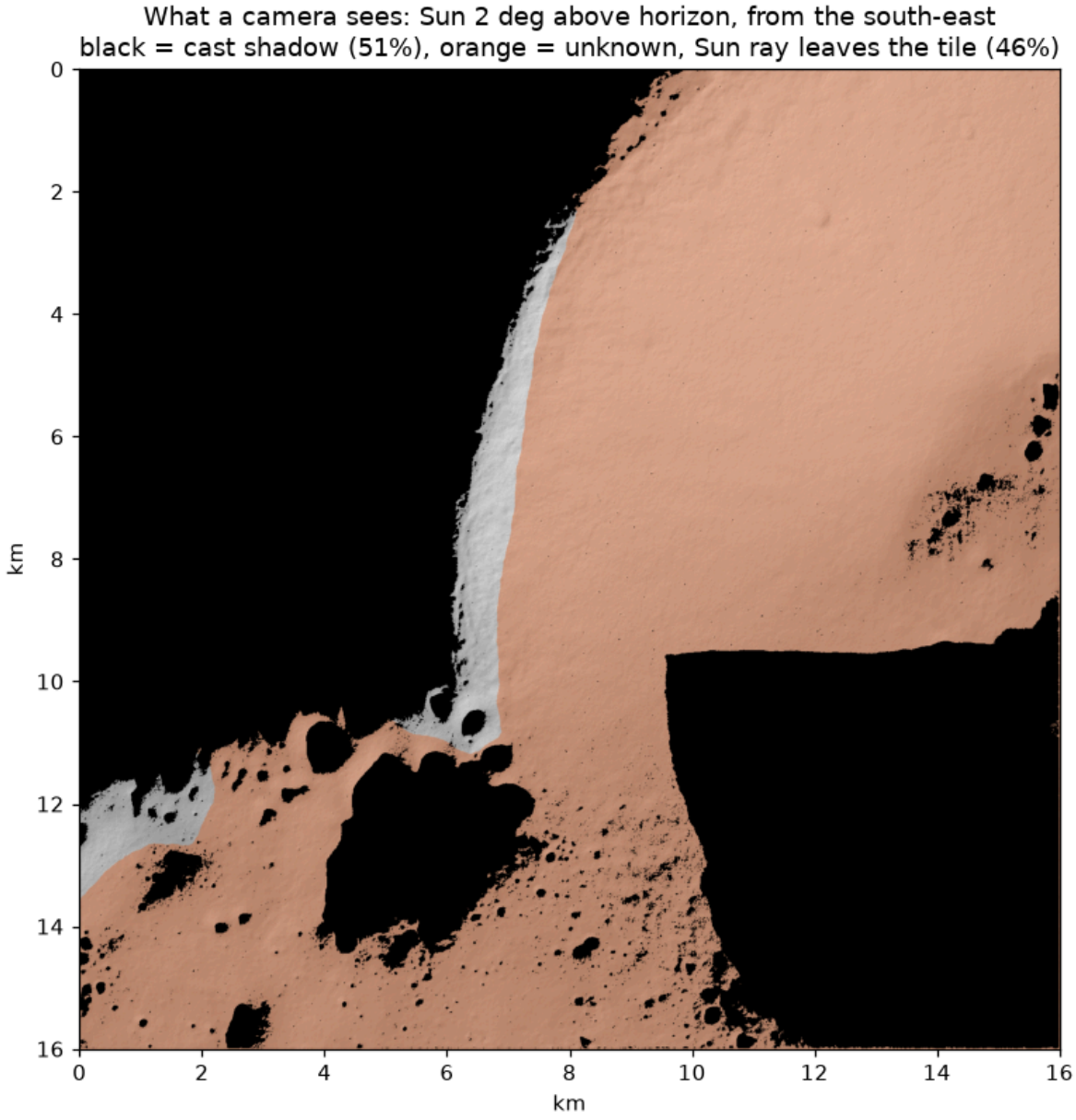
- Along one track across the rim the ground falls 4.5 km over about 9 km, the kind of altitude change that tripped HAKUTO-R.
- With the Sun 2° above the south-east horizon, at least 51% of the tile is in cast shadow.
- **Finding:** another 46% cannot be classified from this tile alone, because at 2° terrain up to about 130 km away can cast shadows. Shadow modelling therefore needs the wider horizon map listed above.



Elevation (left) and slope (right). Most of the crater's inner wall is steeper than the 10° Nova-C landing limit.



One track across the tile: the ground rises to the rim, then falls 4.5 km to the crater floor.



Sun 2° above the horizon from the south-east. Black is cast shadow; orange cannot be decided without terrain beyond the tile.

4. Machine learning components

Model	Input	Output	Training data
Fault-or-terrain classifier (core)	Innovation, DEM height spread nearby, slope, map error, recent residuals	Real terrain or sensor fault	Monte Carlo runs with injected faults

Model	Input	Output	Training data
Crater detector	Rendered camera image	Crater positions for optical navigation	Rendered images; crater positions known from procedural generation
Camera reliability model	Rendered image, Sun angle, terrain roughness	Expected camera error	Detector error measured in simulation
RL landing controller (experiment)	Estimated state	Thrust commands	Thousands of procedural worlds (domain randomisation)

The split follows practice in the literature: ML for perception and prediction (crater detection, terrain uncertainty), analytical rules for the accept/reject decision so each rejection can be explained. The RL controller is tested flying on the filter's estimate, not the true state, so it measures navigation and control together.

5. Experiments and evaluation

1. **Failure-mode scenarios**, one per recent mission:
   a. cliff crossing at the Shackleton rim (HAKUTO-R);
   b. descent into shadow with a noisy altimeter (IM-2);
   c. delayed altimeter readings (ispace M2);
   d. altimeter dropout (IM-1).
2. **Comparison of judges:** every judge in section 1 runs on the same trajectories.
3. **Site change:** tune and train on Site01 and procedural terrain, then test on Site04, which is never seen in tuning; plus a mid-descent divert and a deliberately wrong or missing map.
4. **Lighting sweep:** Sun elevation 1.5°, 2° and 3° at several azimuths.
5. **Monte Carlo:** about 500 runs per scenario with fixed seeds, headless where possible.

Metric	Unit	Better
Landing success (upright, vertical speed and tilt within limits)	% of runs	Higher
Altitude estimation error	RMS, m	Lower
False rejection of valid altimeter readings	%	Lower
Missed detection of injected faults	%	Lower

Metric	Unit	Better
Camera weight while in shadow	ratio	Lower
Fuel remaining at touchdown	kg	Higher

Fair-testing rules (fixed before any experiment runs)

1. **Imperfect map:** the lander's onboard map differs from the simulated "real" Moon (added boulders and small craters, small shifts, noise drawn from NASA's error maps), so the map-based rule cannot win by reading the answer key.
2. **Count both mistakes:** real terrain wrongly rejected *and* faulty readings wrongly believed. A method must do well on both.
3. **Hard cases:** include faults that happen exactly at cliffs, and faults together with cliffs.
4. **Unseen site:** tune on Site01 and procedural terrain; test on Site04, never used for tuning.
5. **Many runs:** about 500 per scenario, reporting success rates with error margins.
6. **Same conditions:** every method sees the same descents, noise and faults.
7. **Fast, repeatable core:** the hundreds of test landings run in a simple Python simulation (numbers only, fixed random seeds); Unity replays selected landings in 3D for the demonstration.
8. **Real-data check:** replay the measured altitude curve from ispace's HAKUTO-R debriefing through each method.

6. Work plan for a team of three

Member	Owns
A: World	DEM import, procedural terrain, horizon map, lighting, floating origin, lander physics and legs
B: Navigation	Sensors, fault injection, EKF and trust layer, guidance, HUD
C: ML and evaluation	Python bridge, data generation, classifier, crater detector, RL experiment, Monte Carlo and plots

Semester plan, with a working result at each stage:

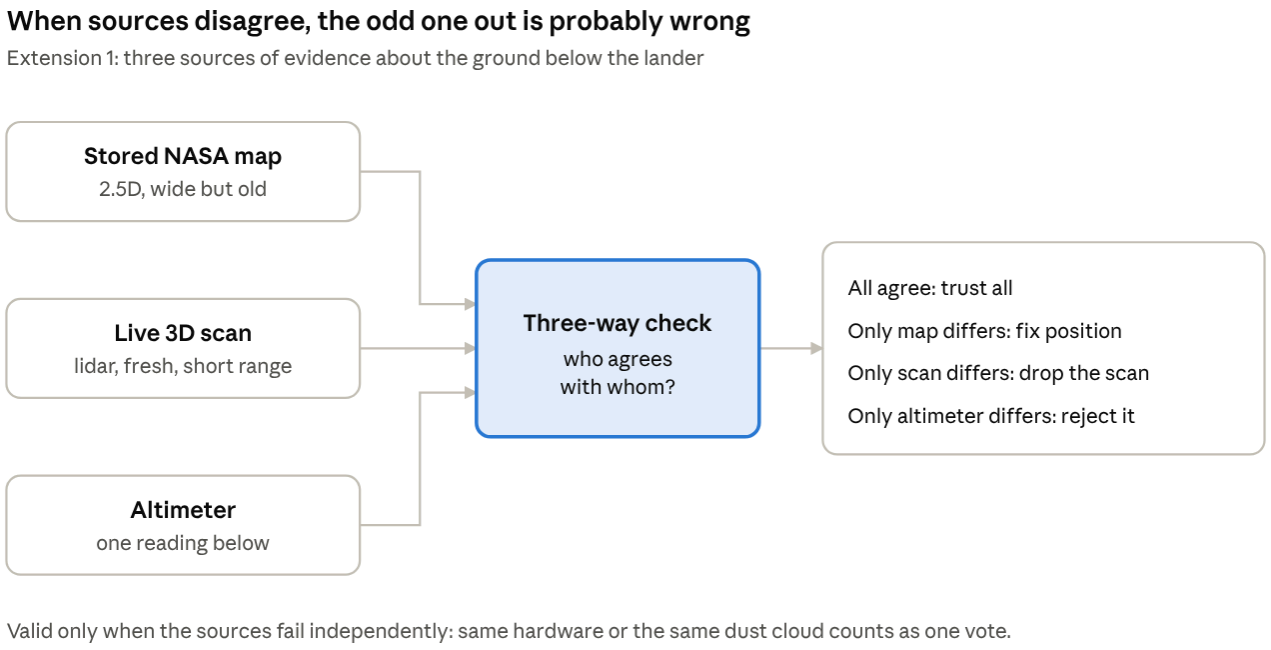
Weeks	Work	Result
1–4	Python test bench: lander, altimeter, NASA terrain, the standard fixed rule	HAKUTO-R-style failure reproduced
5–8	The four-step rule and the fair-testing rules	Main results (H1)
9–11	Shadows and camera, unseen site, HAKUTO-R replay, "when does it help" sweep	H2, H3, H4
12–14	Unity 3D replay of key landings, report, presentation	Demo and final report
If time	Extension 1 (live 3D scan) and the ML fault classifier	Bonus comparisons

7. Scope and extensions

The project addresses sensor trust under terrain and lighting: cliffs, shadows, sensor faults and site changes. Boulder avoidance, engine failure and touchdown leg dynamics beyond simple tip-over are out of scope. Plume dust is the first planned extension: in simulation it is extra camera and lidar noise below about 30 m, and the trust layer treats it as one more condition, following ARC-LIGHT's approach of estimating how bad conditions are before trusting a sensor.

8. Extension 1: live 3D scan

The stored NASA map is wide but old and coarse (5 m); it cannot show boulders or tell the lander exactly where it is on it. A live 3D scan is fresh and detailed but short-range. Combining them gives a third source of evidence, so when sources disagree the odd one out can be identified.



Extension 1 · three sources and the three-way check

How the scan is built

1. **Scan:** a scanning lidar fires a grid of beams (for example 64 x 64) and records each distance. Lidar brings its own light, so it works in shadow. In Unity each beam is one raycast from the lander, with added noise and dropouts.
2. **Points:** the lander's estimated position and tilt turn each distance into a 3D point.
3. **Height patch:** the points are averaged into a grid of heights, the same format as NASA's map, with a confidence per square (many agreeing points: high; few or scattered: low).
4. **Match:** the patch is slid over the stored map near the estimated position, and the best-fitting shift corrects the lander's position. One clear best fit means a confident correction; many similar fits (flat, featureless ground) mean low confidence.
5. **Mix:** where both cover the same square, the heights are combined as a weighted average, giving more weight to the more certain source. Stored-map certainty comes from NASA's error file; scan certainty from the point count and agreement.

Who to trust

Altimeter vs stored map	Altimeter vs live scan	Most likely meaning	Action
Agree	Agree	All fine	Trust all
Disagree	Agree	Old map wrong here, or position off	Trust altimeter and scan; correct position

Altimeter vs stored map	Altimeter vs live scan	Most likely meaning	Action
Agree	Disagree	Scan problem (dust, noise, range)	Distrust the scan
Disagree	Disagree	Altimeter fault	Reject the altimeter

This works only if the sources fail independently. If the altimeter and scanner share hardware, or engine dust blinds both, their agreement counts once. The simulation therefore gives them different error behaviour and tests cases where both fail together, following the use of dissimilar altimeters on Chandrayaan-3 and IM-3.

When it is active: above a few kilometres the scan cannot reach the ground, so the stored map and the four-step rule work alone. From a few kilometres down, matching corrects position. In the last few hundred metres, the mixed map and three-way check run together, and the scan can also spot rocks the stored map misses.

Experiment: compare landing success with the four-step rule alone against the rule plus the live scan, on the same descents. Estimated effort is about 3–4 weeks, so it is scheduled after the core results.

Key references

Adams et al. (2025), AIAA SciTech, NTRS 20240014100 · Barker et al. (2021), *Planetary and Space Science* 203 · Bertone et al., *Planetary Science Journal*, doi:10.3847/PSJ/ae5b70 · Karlgaard et al. (2025), NTRS 20250010290 · ispace HAKUTO-R M1 debriefing (2023). Full list in the Literature Review document.